Experiment 10

Student Name: Abhishek Sharma **UID: 23BCS12330**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:22/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design a **Food Delivery Platform (similar to Swiggy, Zomato)** that enables users to browse restaurants, place orders, track deliveries in real-time, and make secure payments with high availability and low latency.

2. **Objective**:

- To design a scalable architecture for a food delivery system
- To enable real-time order tracking using location services
- To support order placement, cancellation, and status updates
- To ensure reliability in order processing and payment handling
- To implement delivery partner assignment and tracking
- To optimize performance using caching and asynchronous processing.

3. **Tools Used:**

- **Frontend (ReactJS, HTML, CSS, JavaScript)**
  - Built responsive UI for browsing restaurants and placing orders
  - Integrated APIs for dynamic menu and order tracking
- **Backend (Node.js / Java / Spring Boot)**
  - Developed REST APIs for users, orders, delivery, and payments
  - Implemented business logic for order lifecycle
- **Database (PostgreSQL / MySQL)**
  - Managed transactional data (orders, users, payments)
  - Ensured ACID properties for order consistency
- **Redis**
  - Cached frequently accessed data (menus, restaurant list)
  - Reduced latency for read-heavy operations
- **Message Queue (Kafka / RabbitMQ)**
  - Used for asynchronous processing (order → delivery assignment)
  - Improved scalability and fault tolerance
- **Maps API (Google Maps / Mapbox)**
  - Used for real-time delivery tracking and ETA calculation
- **Deployment (AWS / GCP, Docker, Load Balancer)**
  - Cloud-based scalable infrastructure
  - Containerized services for easy deployment

4. System Requirements:

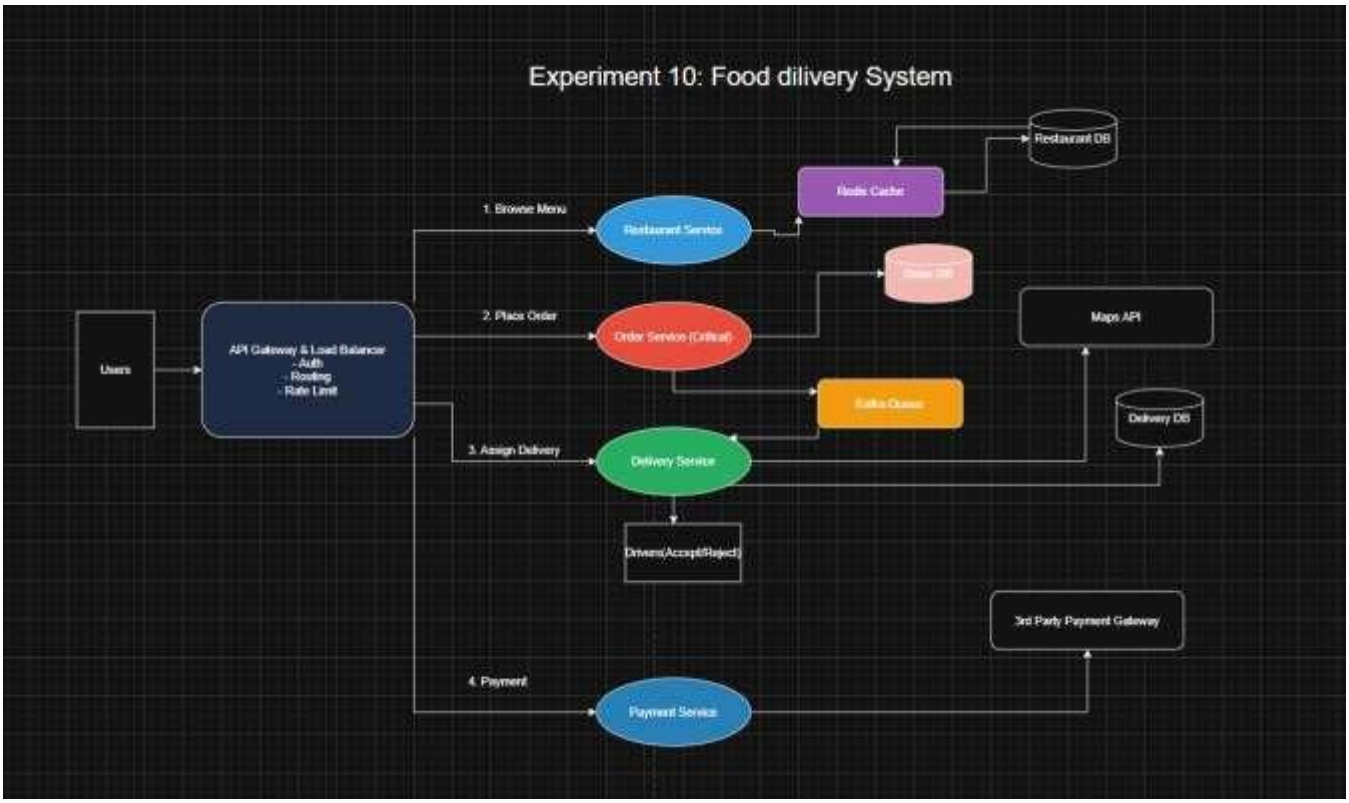
A. Functional Requirements

- Users should be able to register and login
- Users should be able to browse restaurants and menus
- Users should be able to add items to cart and place orders
- The system should assign a delivery partner automatically
- Users should be able to track order status in real-time
- Users should be able to make payments (online / COD)
- Users should be able to cancel orders (before preparation stage)
- Restaurants should be able to accept/reject orders
- Delivery partners should be able to accept/decline delivery requests
- Users should be able to rate and review orders.

B. Non-Functional Requirements

- **Scalability**
  System should support **millions of users and thousands of concurrent orders**, especially during peak hours. **CAP Theorem**
- **Consistency + Availability**
  Strong consistency → Order & Payment
  Eventual consistency → Tracking & notifications
- **Latency:**
  Order placement: **< 200 ms**
  Menu fetch: **< 100 ms**
  Tracking updates: **real-time (~1–2 sec delay)**
- **Reliability:**
  No duplicate orders
  Payment must be processed exactly once

5. HLD:



6. Scalability Solution

- Use **WebSockets** for real-time stock price updates
- Use **API Gateway + Load Balancer** to distribute traffic
  Use **Redis caching** for menu and restaurant data
  Use **Kafka (event-driven architecture)** for order processing
  Use **microservices architecture**:
- Order Service
- Delivery Service
- Payment Service
- Restaurant Service
- Use **database sharding** for large-scale data
  Use **horizontal scaling** for services
  Optimize read-heavy operations via caching
- Optimize read-heavy operations using caching

7. Learning Outcomes (What I Have Learnt)

- Understood architecture of large-scale food delivery systems
- Learned how to design microservices-based applications
- Understood importance of **asynchronous processing (Kafka)**
- Learned real-time tracking using location services
- Understood trade-offs between **consistency vs availability**
- Gained knowledge of scaling techniques (cache, queue, load balancing)
- Learned how real-world systems handle peak traffic efficiently